Direct Manipulation Matrix Sorting Application

Implementation Process and Design Report

CSCI 626 Midterm Examination

React Vite Tailwind CSS SVG

This report explains how the submitted web application was created and how its drag gestures sort a 30 by 40 numerical matrix. The implementation displays all 1,200 values at once and performs row, column, and combined sorting without buttons, menus, sliders, checkboxes, tooltips, or other interface widgets.

Prepared from the verified project source and production build

Report Contents

1. Project objective and requirement compliance
2. Technology choice and project creation
3. Application structure and matrix data
4. Direct manipulation and gesture classification
5. Sorting algorithms and visual transitions
6. User instructions and test procedure
7. Engineering practices and references

Project Objective

The application turns the matrix itself into the control surface. A user presses a cell and drags in the direction that expresses the required operation. The starting cell supplies the row and column used as sort keys. This keeps the interaction inside the required matrix area and removes the need for conventional widgets.

The values remain attached to their original rows and columns. Sorting changes the displayed order through rowOrder and colOrder arrays; it does not overwrite the underlying 30 by 40 data array. This separation makes the behavior easier to understand, animate, and test.

Requirement Compliance

Requirement	Implementation
30 rows and 40 columns	Constants ROWS = 30 and COLS = 40 create exactly 1,200 cells.
Numerical values	Every cell receives an integer from 0 through 99.
Sort rows	A vertical drag reorders complete rows using the starting column as the key.
Sort columns	A horizontal drag reorders complete columns using the starting row as the key.
Sort both axes	A diagonal drag performs the row sort and column sort from the same starting cell.
Ascending and descending	Down and right mean ascending; up and left mean descending.
No UI widgets	The page contains instructions, the matrix, and an SVG feedback arrow only.
Instructions on the page	A short explanation above the matrix describes every gesture and direction.

Technology Choice

The prompt gives D3.js as an example rather than a required library. This implementation uses React for component state and rendering, Vite for development and production builds, Tailwind CSS for utility styling, and inline SVG for the drag arrow. These browser technologies are sufficient because the visualization needs direct cell positioning and pointer input rather than scales, axes, or chart layouts.

- React stores the row order, column order, selected cell, and drag line as component state.
- Vite provides the development server and creates the optimized production bundle.
- Tailwind CSS supplies concise layout, typography, color, and transition classes.
- Pointer Events provide one interaction model for mouse, pen, and touch input.
- SVG draws a direction arrow without adding a button or another widget.

Project Creation

The following sequence recreates the project from an empty working directory.

8. Create a Vite project with the React template and enter the new matrix-sort directory.
9. Install React, React DOM, Vite, the React plugin, Tailwind CSS, and the Tailwind Vite plugin.
10. Configure Vite to load both the React and Tailwind plugins.
11. Import Tailwind CSS from src/index.css.
12. Create the React root in src/main.jsx and render the App component.
13. Implement the matrix, gesture recognition, sorting logic, and instructions in src/App.jsx.
14. Run the development server during implementation and create a production build before submission.

```
npm create vite@latest matrix-sort -- --template react
cd matrix-sort
npm install
npm install tailwindcss @tailwindcss/vite
npm run dev
npm run build
```

The submitted package.json already defines dev, build, and preview scripts. The verified build completed successfully with 31 transformed modules and generated the production files in the dist directory.

Project Structure

File	Purpose
index.html	Defines the browser document, the root element, and the JavaScript module entry point.
src/main.jsx	Creates the React root, imports the stylesheet, and renders App.
src/App.jsx	Contains the matrix model, gesture handlers, sorting operations, SVG feedback, and cell rendering.
src/index.css	Imports Tailwind CSS.
vite.config.js	Registers the React and Tailwind Vite plugins.
package.json	Records dependencies and development, build, and preview commands.

Matrix Data Model

The application defines the dimensions and cell geometry once. The range helper creates integer index sequences. makeMatrix then creates 30 arrays, each containing 40 random integers. useMemo calls makeMatrix only for the initial component instance, so normal interaction does not replace the values with a new random matrix.

```
const ROWS = 30;
const COLS = 40;

const range = (n) => Array.from({ length: n }, (_, i) => i);

function makeMatrix() {
  return range(ROWS).map(() =>
    range(COLS).map(() => Math.floor(Math.random() * 100))
  );
}
```

The data array stores the values. Two separate state arrays store the current display order. rowOrder begins as 0 through 29, and colOrder begins as 0 through 39. Sorting copies and reorders these index arrays, which preserves the original values and row or column membership.

```
const data = useMemo(makeMatrix, []);
const [rowOrder, setRowOrder] = useState(range(ROWS));
const [colOrder, setColOrder] = useState(range(COLS));
```

Cell Rendering and Visual Encoding

The component maps across the original data array and creates 1,200 absolutely positioned div elements. The rowPos and colPos lookup objects translate original row and column indexes into their current screen positions. CSS transforms move each cell to its ordered position. The transition class animates a sort for 500 milliseconds so the user can see the rows and columns move.

Blue lightness encodes cell magnitude: larger values appear darker. During a drag, the selected cell turns yellow and the complete key row and key column use a yellow scale. White cell borders preserve the grid structure, and high values use white text for contrast.

Sorting the matrix by direct manipulation

Press on a cell, drag, and release. The pressed cell is highlighted together with its row and column, because they supply the sort keys. Whole rows and whole columns are reordered; the numbers inside a row or column are never rearranged.

- **Drag down / up:** reorder the **rows** by the values in the pressed cell's column (down = ascending, up = descending).
- **Drag right / left:** reorder the **columns** by the values in the pressed cell's row (right = ascending, left = descending).
- **Drag diagonally:** do both at once. The vertical part sets the row order and the horizontal part sets the column order. Both sorts use the row and column of the cell you originally pressed.

The arrow color shows which sort will happen. Darker cells have higher values. Drags shorter than a few pixels are ignored.

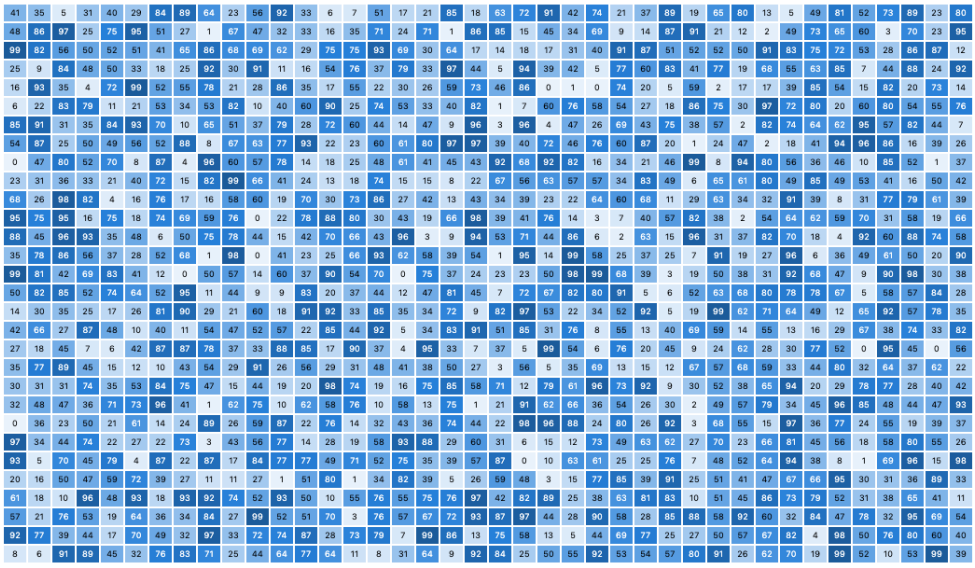


Figure 1 Initial application view with instructions and all 1,200 matrix values

Direct Manipulation Design

The user performs each command directly on a matrix cell. The gesture begins with the sort key and ends with the desired direction. Vertical motion changes row order, horizontal motion changes column order, and diagonal motion changes both. This design follows the direct manipulation principle of acting on visible objects with immediate visual feedback.

Gesture	Sort operation	Direction result	Feedback color
Drag down	Rows by starting column	Ascending	Blue
Drag up	Rows by starting column	Descending	Blue
Drag right	Columns by starting row	Ascending	Green
Drag left	Columns by starting row	Descending	Green
Drag diagonally	Rows and columns	Each axis follows its drag sign	Purple

Gesture Classification

classifyGesture compares the absolute horizontal and vertical distances. A drag shorter than 15 pixels is ignored, which prevents an ordinary click or small hand movement from causing a sort. A gesture is diagonal when its smaller axis is at least half the length of its larger axis. All other gestures are classified according to their dominant axis.

```
function classifyGesture(dx, dy) {
  const ax = Math.abs(dx);
  const ay = Math.abs(dy);
  if (Math.max(ax, ay) < MIN_DRAG) return null;
  const isDiagonalGesture =
    Math.min(ax, ay) / Math.max(ax, ay) >= DIAGONAL_RATIO;
  if (isDiagonalGesture) return 'diagonal';
  return ay > ax ? 'vertical' : 'horizontal';
}
```

The arrow uses the same classification function while the pointer is moving. Its color therefore previews the operation that will occur when the pointer is released.

Sorting the matrix by direct manipulation

Press on a cell, drag, and release. The pressed cell is highlighted together with its row and column, because they supply the sort keys. Whole rows and whole columns are reordered; the numbers inside a row or column are never rearranged.

- **Drag down / up:** reorder the **rows** by the values in the pressed cell's column (down = ascending, up = descending).
- **Drag right / left:** reorder the **columns** by the values in the pressed cell's row (right = ascending, left = descending).
- **Drag diagonally:** do both at once. The vertical part sets the row order and the horizontal part sets the column order. Both sorts use the row and column of the cell you originally pressed.

The arrow color shows which sort will happen. Darker cells have higher values. Drags shorter than a few pixels are ignored.

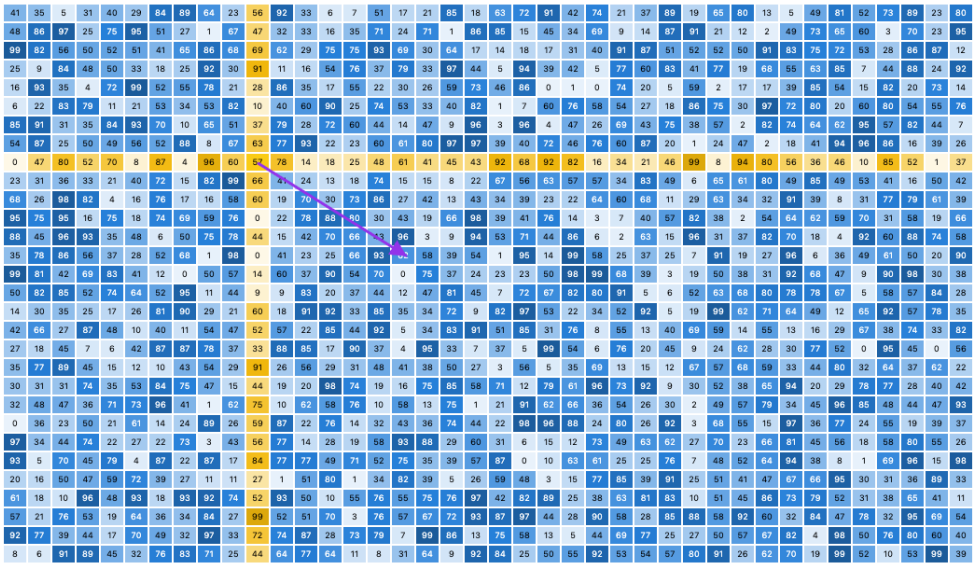


Figure 2 Diagonal drag with a purple arrow and highlighted key row and column

Pointer Event Sequence

The matrix container handles the complete pointer lifecycle. This reduces repeated event listeners and keeps the gesture state in one component.

15. onPointerDown reads the data-row and data-col attributes from the pressed cell.
16. The handler converts the pointer location into coordinates relative to the matrix container.
17. setPointerCapture keeps subsequent pointer events attached to the container while the drag continues.
18. onPointerMove updates the SVG line and causes the row and column highlights to remain visible.
19. onPointerUp calculates dx and dy, classifies the gesture, clears the temporary feedback, and runs the requested sort.
20. onPointerCancel clears the same temporary state if the browser cancels the interaction.

```
if (gesture === 'vertical' || gesture === 'diagonal')
  sortRows(s.col, dy > 0);

if (gesture === 'horizontal' || gesture === 'diagonal')
  sortCols(s.row, dx > 0);
```

The signs of dy and dx encode ascending or descending order. A positive vertical displacement means down and therefore ascending rows. A positive horizontal displacement means right and therefore ascending columns.

Row Sorting

sortRows receives the original column index of the pressed cell and a Boolean direction. It copies the current row order, compares the values from the chosen column, and updates the row order state. The multiplier is 1 for ascending order and negative 1 for descending order.

```
const sortRows = (keyCol, asc) =>
  setRowOrder((order) =>
    [...order].sort((a, b) =>
      (asc ? 1 : −1) * (data[a][keyCol] - data[b][keyCol])
    )
  );
```

Column Sorting

sortCols follows the same pattern but compares column indexes by reading values from the original row of the pressed cell. Only colOrder changes, so complete columns move together.

```
const sortCols = (keyRow, asc) =>
  setColOrder((order) =>
    [...order].sort((a, b) =>
      (asc ? 1 : −1) * (data[keyRow][a] - data[keyRow][b])
    )
  );
```

Combined Row and Column Sorting

A diagonal release calls both functions. The starting row and column are saved before either sort begins, so the first state update cannot change the key used by the second operation. React applies the two state changes and renders the new row and column positions together.

Visual Reordering

The rowPos and colPos objects invert the order arrays. Each original index maps to its current visible position. The transform expression multiplies those positions by the fixed cell width and height. React keeps each cell identity stable with a key made from its original row and column, so the browser animates existing cells instead of replacing them.

```
const rowPos = useMemo(
  () => Object.fromEntries(rowOrder.map((r, i) => [r, i])),
  [rowOrder]
);

transform: `translate(${colPos[c] * CELL_W}px,
                      ${rowPos[r] * CELL_H}px)`
```

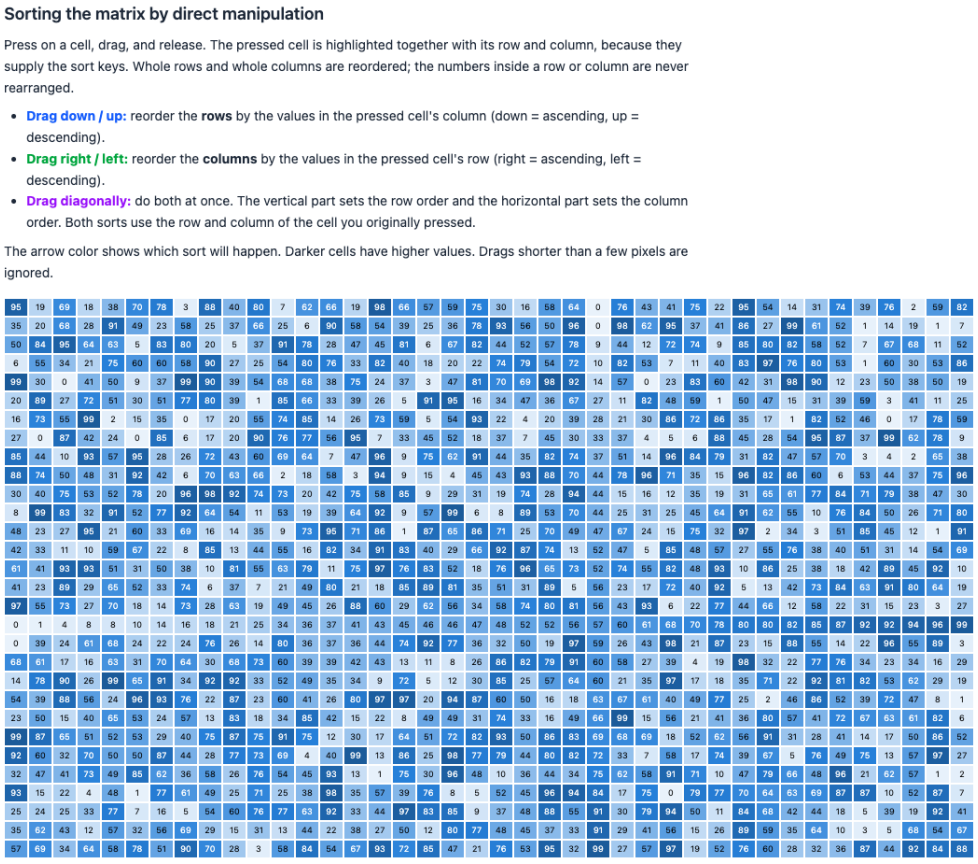


Figure 3 Matrix after the diagonal sort has reordered both axes

Instructions Presented to the User

The page tells the user to press a cell, drag, and release. It explains that the selected row and column provide the sort keys and that the values inside those groups are not independently rearranged. The three gesture descriptions use the same blue, green, and purple colors as the feedback arrow. This creates a visible connection between the written instructions and the live gesture preview.

Because the assignment penalizes every displayed widget, the design contains no button, select menu, slider, checkbox, tooltip, text field, legend control, or reset control. The text and matrix provide all required information and interaction.

Testing and Verification

The implementation can be checked with the following procedure.

21. Run npm install from the matrix-sort directory if dependencies are not already installed.
22. Run npm run dev and open the local address shown by Vite.
23. Confirm that the matrix contains 30 visible rows and 40 visible columns.
24. Drag downward from any cell and confirm that the values in its original column become ascending from top to bottom.
25. Drag upward and confirm that the same row sort is descending.
26. Drag right from any cell and confirm that the values in its original row become ascending from left to right.
27. Drag left and confirm that the same column sort is descending.
28. Drag diagonally in each direction and confirm that both axes change according to the horizontal and vertical signs.
29. Make a movement shorter than 15 pixels and confirm that no sorting occurs.
30. Run npm run build and confirm that Vite produces the dist directory without errors.

Verification performed for this report confirmed a successful production build. A browser snapshot also exposed all 1,200 numerical cell nodes, and a diagonal pointer gesture produced the expected purple arrow, key-line highlighting, and final matrix reordering. The only browser console error was a missing optional favicon, which does not affect matrix behavior.

Software Engineering Practices

- Constants define dimensions, geometry, thresholds, and gesture colors in one location.
- Functions have focused responsibilities for data generation, gesture classification, row sorting, column sorting, pointer conversion, and drag cleanup.
- State names describe the information they hold: rowOrder, colOrder, selected, and line.
- The code copies arrays before sorting, which avoids mutating React state.
- A targeted comment explains why diagonal sorting must retain the originally pressed row and column.
- Pointer cancellation and minimum drag handling prevent unintended actions.
- Stable cell keys and transform transitions preserve visual continuity during sorting.

Limitations and Possible Improvements

The values are random on each full page load, so a screenshot from one session cannot be compared numerically with a later reload. Equal values depend on the JavaScript engine's stable array sort behavior. The fixed 26 by 20 pixel cells also assume a desktop-width viewport; a narrow display may require horizontal scrolling. These constraints do not prevent the required sorting operations.

A later version could add keyboard gestures or screen-reader instructions while preserving the prohibition on visible widgets. Any such extension should retain direct manipulation as the primary interaction and should not add controls that violate the assignment.

References

1. Shneiderman, B. 1983. Direct Manipulation A Step Beyond Programming Languages. Computer 16 8 57 to 69. DOI 10.1109 MC 1983 1 654 471. https://doi.org/10.1109/MC.1983.1654471

2. React documentation for useState. https://react.dev/reference/react/useState

3. React documentation for useMemo. https://react.dev/reference/react/useMemo

4. React documentation for useRef. https://react.dev/reference/react/useRef

5. Vite getting started guide. https://vite.dev/guide/

6. Vite production build guide. https://vite.dev/guide/build

7. Tailwind CSS installation with Vite. https://tailwindcss.com/docs/installation/using-vite

8. MDN documentation for Element setPointerCapture. https://developer.mozilla.org/en-US/docs/Web/API/Element/setPointerCapture

Source Note

The gesture mapping and application code are implemented in the submitted project rather than copied from a third-party example. The references above document the direct manipulation concept and the framework and browser APIs used to build the application.